



CarenR vs Ripper - differences

Status	Done
Last edited time	April 4, 2026 6:30 PM

- ☐ Create a summary comparassion to use in the thesis
- ☐ Create a infographic here

Algorithm definition

Practical recommendation

Use **two different input versions**:

For CarenR

- use the **discretized dataset**
- because CarenR, in your workflow, was designed around that representation

For RIPPER

- use the **original variables**
- let RIPPER handle numeric predictors internally by finding thresholds such as:

```
temperature <= 18.7  
rainfall > 42.3
```

That is usually the fairest setup.

Why this is the better choice

RIPPER is a **classification rule learner** and can naturally work with **continuous predictors** by searching split points during rule induction. So if you discretize everything beforehand, you may:

- lose information
- force arbitrary cut points
- make RIPPER less flexible than it actually is

If your goal is to compare **algorithms fairly as they are meant to be used**, then:

- **CarenR** → discretized input
- **RIPPER** → original numeric input

Important exception

If your goal is not an algorithmic comparison, but rather a **strict comparison under exactly the same feature space**, then you could also run RIPPER on the discretized variables.

That would answer a different question:

“How do CarenR and RIPPER behave when both receive the same transformed input?”

So in practice, the strongest thesis setup is often:

Recommended experiment design

Experiment A — Native setup

- CarenR on discretized features
- RIPPER on original features

 This compares the methods in their **natural operating mode**.

Experiment B — Same-feature comparison

- CarenR on discretized features
- RIPPER on the same discretized features

 This compares the methods under the **same representation**.
That gives you a much stronger discussion section.

! Very important point: target variable

RIPPER is **classification**, not regression.

So if your target is still continuous wine production, you **must discretize the target** for RIPPER.

Example:

- low production
- medium production
- high production

This is separate from predictor discretization.

So the setup becomes:

- predictors: preferably original continuous values
- target: discretized categories

Setup

My strong recommendation is:

Do not add a heavy extra discretization layer for RIPPER by default

For **RIPPER**, the best primary setup is usually:

- **predictors**: keep them in their **original continuous form**
- **target**: discretize it into a **small number of classes**

That is the most natural use of RIPPER/JRip, because the Weka/RWeka pipeline can handle numeric predictors and create threshold-based antecedents internally, while supervised discretization is available when you

explicitly want it. RWeka also provides a supervised Discretize filter based on the **MDL method of Fayyad and Irani (1993)**.

Since your thesis workflow already separates **CarenR rule discovery**, **RIPPER rule discovery**, and **ML model comparison**, using RIPPER as a second rule learner makes most sense when it is allowed to operate in its own natural regime, rather than forcing it into exactly the same discretized representation as CarenR.

So what should you do?

Primary experiment for RIPPER

Use:

- **original predictors**
- **discretized target**

Secondary robustness experiment

Also test:

- **discretized predictors**
- **same discretized target**

This gives you both:

- a **native RIPPER benchmark**
- a **same-feature-space comparison** against CarenR

That is a much stronger thesis design than choosing only one setup.

How many target classes should you use?

Given your case, I would **start with 3 classes**.

For a dataset covering **1933–2022**, you likely have on the order of roughly 90 annual observations, which is not large for rule induction. With small samples, too many classes fragment the data quickly and produce unstable, overly specific rules.

Strong recommendation

Use:

- **3 classes** as the default:
 - **Low**
 - **Medium**
 - **High**

This is usually the best balance between:

- interpretability
- enough observations per class
- stable rules
- meaningful comparison with CarenR and tree models

Only test 4 classes if:

each class still has a reasonable number of observations. For your sample size, I would avoid 4 classes unless the smallest class still has at least around **15 observations**. Below that, RIPPER can become too unstable and start generating brittle rules.

Avoid:

- **5+ classes**
 - very fine target discretization
 - rare-class targets unless that is your explicit scientific question
-

How should you choose the cut points?

There are three reasonable approaches, ranked from strongest to weakest for your use case:

1. Domain-based cut points

Best if you have agronomic meaning.

Example:

- low production = poor yield years
- medium = normal years
- high = exceptional years

This is strongest for thesis interpretability, because the classes have real meaning.

2. Supervised MDL discretization

Best if you want a data-driven classification target.

If you do not have strong domain thresholds, use **supervised discretization**, ideally **MDLP / Fayyad-Irani**, because it is designed to find informative class boundaries rather than arbitrary equal-width bins. RWeka exposes exactly this type of discretization.

3. Quantile-based classes

Acceptable fallback.

If domain thresholds are not available and MDLP gives awkward or unstable splits, use:

- terciles for **3 classes**
- quartiles for **4 classes**

For your thesis, terciles are usually safer than quartiles because they preserve sample size per class.

Should you discretize the predictors too?

My recommendation:

Not for the main RIPPER run.

RIPPER/JRip already stores **numeric split points** internally, and Weka's JRip implementation includes explicit support for numeric antecedents with learned split points.

So if you discretize predictors beforehand, you may:

- lose information
- force artificial thresholds
- reduce RIPPER's flexibility
- make the comparison less about the learner and more about the preprocessing

When discretizing predictors is still useful

Use a second run with discretized predictors only if you want to answer:

“How much of the performance difference comes from the algorithm itself, and how much comes from the feature representation?”

That is a valid and useful thesis sensitivity analysis.

If you insist on discretizing predictors for RIPPER, what should you use?

Use **supervised MDL discretization**, not manual “higher discretization.”

That means:

- not equal-width
- not too many intervals
- not arbitrary fine bins

RWeka’s supervised Discretize() uses the **Fayyad-Irani MDL** approach, which is still a very solid choice for classification-oriented preprocessing.

Good rule for interval count

For your sample size, I would keep most predictors at:

- **2 to 4 intervals**
- preferably **2 or 3**
- only allow 4 if the variable clearly has enough support across bins

A practical constraint:

- **no interval should have fewer than ~8–10 observations**
- ideally **10+**

If bins get smaller than that, the rules will become fragile very quickly.

Best current recommendation for your thesis

Recommended experimental design

Setup A — Main RIPPER benchmark

- **original predictors**
- **3 target classes**

Setup B — Sensitivity on target granularity

- **original predictors**
- **4 target classes** only if class sizes remain healthy

Setup C — Same-feature comparison with CarenR

- **discretized predictors**
- **same target classes as Setup A**

This gives you a very credible analysis:

- native RIPPER

- effect of target discretization
- effect of predictor discretization
- fair comparison with CarenR

My strongest practical suggestion

If I had to choose only one setup today, I would use:

Final default choice

- **RIPPER with original predictors**
- **target discretized into 3 classes**
- target cut points chosen by:
 1. **domain thresholds**, if available
 2. otherwise **MDLP**
 3. otherwise **terciles**

And I would add **one sensitivity run** using the discretized predictor set from CarenR.

That is the most defensible setup academically and the most likely to produce rules that are both stable and interpretable.

If you want, I'll turn this into an **R execution plan** with the exact RWeka::JRip workflow and the recommended preprocessing steps for your dataset.

An important one versus the other for my thesis

In my case, where the target is continuous, depending on the rules I find with CarenR, I would expect the rules to align somehow. But CarenR could have an advantage if the rules are good, because it is not limited to a classification target.

Yes — your reasoning is broadly correct, and it is actually one of the most important conceptual distinctions in your thesis.

Table of scenarios

Scenario	Predictors	Target	Purpose	Main advantage	Main risk	Potential Recommendation
A. Native RIPPER baseline	Original continuous predictors	3 discretized classes	Use RIPPER in its natural classification setup	Preserves predictor information; lets RIPPER find its own thresholds	Target discretization may lose some signal	Main RIPPER experiment
C. Same-feature comparison with CarenR	Discretized predictors used for CarenR	Same discretized target as A	Compare algorithms under the same feature representation	Fairer comparison of learner behavior	RIPPER may lose flexibility from forced discretization	Strong secondary experiment

Scenario	Predictors	Target	Purpose	Main advantage	Main risk	Potential Recommendation
E. CarenR main setup	Discretized predictors	Continuous target or target treatment native to CarenR	Main rule discovery method of the thesis	Can discover rules without forcing strict class labels	Depends strongly on discretization quality and rule quality	Main thesis setup
F. Tree-based ML benchmark	Original continuous predictors	Continuous target	Predictive benchmark	No need to discretize target; strong predictive baseline	Less interpretable than rules	Necessary benchmark
G. Rule-based prediction from CarenR rules	Rules from CarenR	Continuous target reconstructed from covered instances / neighbors	Test if discovered rules can predict production	Directly aligned with your thesis logic	Prediction pipeline may add variance	Important evaluation block
H. Rule-based prediction from RIPPER rules	RIPPER rules	Class prediction, optionally mapped back to production range or class midpoint	Compare predictive utility of RIPPER rules	Simple, transparent comparison	Limited by target discretization	Useful but secondary
D. Fully discretized RIPPER	Discretized predictors	4 discretized classes	Stress-test highly discretized setup	Maximum comparability with CarenR-style representation	Highest risk of information loss and brittle rules	Optional only
B. RIPPER with finer target granularity	Original continuous predictors	4 discretized classes	Test sensitivity to target discretization	May capture more nuanced production regimes	Class fragmentation; unstable rules with small sample	Only if class sizes remain healthy

Is your reasoning correct?

Yes. The core idea is this:

RIPPER is fundamentally constrained by classification

RIPPER learns rules of the form:

```
IF conditions THEN class = Low/Medium/High
```

So if your original target is **continuous wine production**, RIPPER can only operate after you **convert that target into classes**.

That means RIPPER is learning:

- boundaries between production categories
- not the continuous production process directly

CarenR may have a real conceptual advantage

If CarenR can discover rules while keeping the target in a more continuous or distribution-aware setting, then it is **less constrained** than RIPPER.

So in your case, if the CarenR rules are good, then CarenR may have an advantage because:

- it is not forced to compress the target into a few labels
- it may preserve more information about production magnitude
- it may capture more nuanced patterns in the relationship between meteorology and wine production

That is a perfectly valid thesis argument.

But there is an important nuance

This advantage is **conceptual**, not automatic.

CarenR only has the advantage **if**:

1. the discretization of predictors is meaningful
2. the rules found are stable and interpretable
3. the rule coverage is not too fragmented
4. the resulting rule-based predictions are competitive or at least insightful

So the correct thesis position is not:

“CarenR is better because it handles continuous target.”

It should be:

“CarenR has a potential methodological advantage because it is not restricted to a discretized classification target, whereas RIPPER requires discretization of the response variable. This may allow CarenR to preserve more information about the target and potentially extract richer rule structures, provided that the discovered rules remain stable, interpretable, and useful.”

That is a much stronger academic formulation.

What you should expect empirically

Your expectation that the rules may **align somehow** is also correct.

You may find that:

- RIPPER identifies rules corresponding to **low / medium / high production years**
- CarenR identifies similar meteorological regions, but with more nuanced target behavior

Example:

RIPPER

```
IF spring_temp high AND summer_water low THEN class = Low Production
```

CarenR

```
IF spring_temp high AND summer_water low THEN production tends to be between X and Y,  
with mean around Z
```

So the **direction of the pattern** may align, even if the target treatment differs.

That would actually be a very strong result for your thesis:

- agreement in structure
 - difference in target expressiveness
-

Best interpretation for your thesis

If both methods find similar patterns, you can argue:

- the meteorological relationships are robust
- CarenR provides a richer description because it is not restricted to a categorical target
- RIPPER confirms the main directional structure in a simpler classification setting

If they differ, you can argue:

- discretizing the target may be hiding or distorting part of the continuous relationship
- CarenR may capture effects that RIPPER cannot represent due to class boundaries

Both outcomes are interesting.

Strong thesis conclusion you may later use

You are essentially comparing:

- **CarenR** as a rule discovery method potentially better suited to continuous-response analysis
- **RIPPER** as a strong interpretable classification benchmark
- **tree models** as predictive continuous-response benchmarks

That is a good and coherent design.

My strongest suggestion

Frame RIPPER in your thesis as:

- an **interpretable benchmark rule learner**
- useful to test whether the main structures found by CarenR also appear under a simpler classification formulation

And frame CarenR as:

- the method with potential added value because it is **not inherently limited to a discretized target**

If you want, I can now turn this into a **thesis-ready discussion paragraph** explaining exactly why comparing CarenR and RIPPER is methodologically meaningful.